LAPORAN PRAKTIKUM

APLIKASI BERBASIS PLATFORM

KELAS IF-11-04

Dosen: Cahyo Prihantoro, S.Kom., M.Eng

Nama : Daffa Falih Aqilah

NIM : 2311102137

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM

PURWOKERTO

2025/2026

LINK GIT :

MODUL 1

RUNNING MODUL

1.1. Git

Git adalah salah satu sistem pengontrol versi (Version Control System) pada proyek perangkat lunak yang diciptakan oleh Linus Torvalds. Pengontrol versi bertugas mencatat setiap perubahan pada file proyek yang dikerjakan oleh banyak orang maupun sendiri. Git dikenal juga dengan distributed revision control (VCS terdistribusi), artinya penyimpanan database Git tidak hanya berada dalam satu tempat saja.

1.2. Instalasi JDK

JDK (Java Development Kit) merupakan perangkat yang digunakan untuk melakukan proses kompilasi dari kode java ke bytcode yang dapat dimengerti dan dapat dijalankan oleh JRE (Java Runtime Environment).

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ASUS>java --version
java 23.0.2 2025-01-21
Java(TM) SE Runtime Environment (build 23.0.2+7-58)
Java HotSpot(TM) 64-Bit Server VM (build 23.0.2+7-58, mixed mode, sharing)

C:\Users\ASUS>
```

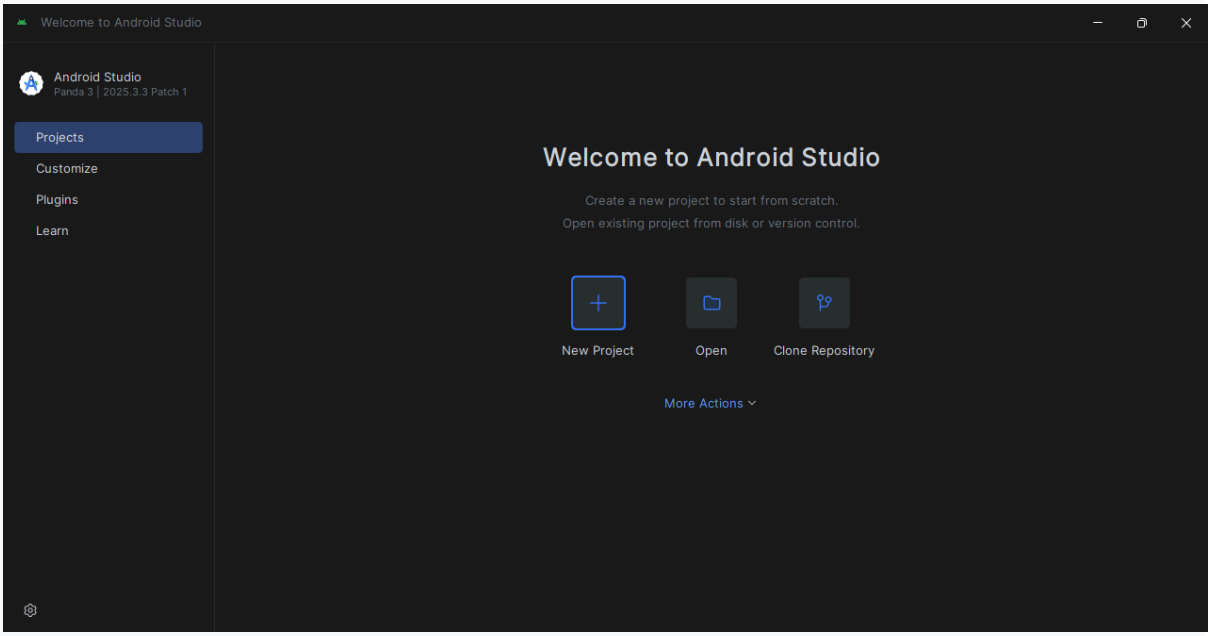
1.3. Instalasi Flutter SDK

Sebelum melakukan instalasi Flutter, kita perlu menyiapkan dan menginstall tools yang dibutuhkan saat pengembangan aplikasi menggunakan Flutter.

```
C:\Users\ASUS>flutter --version

 A new version of Flutter is available!

 To update to the latest version, run "flutter upgrade".

Flutter 3.41.7 • channel stable • https://github.com/flutter/flutter.git
Framework • revision cc0734ac71 (3 weeks ago) • 2026-04-15 21:21:08 -0700
Engine • hash 7a53c052bc4b472cf780b199087e1368e4a9aa8c (revision 59aa584fdf) (17 days ago) • 2026-04-16 02:32:16.000Z
Tools • Dart 3.11.5 • DevTools 2.54.2
```

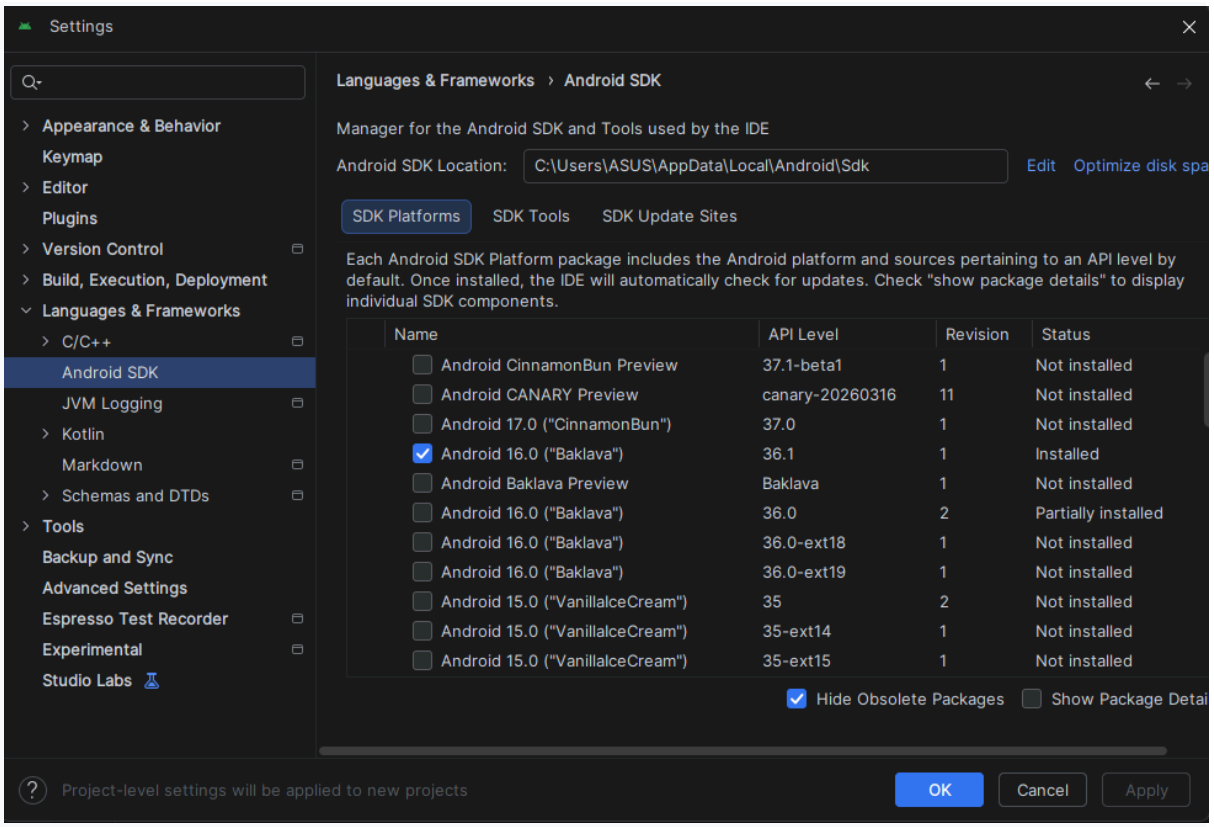
1.4. Instalasi Android Studio

Android Studio merupakan IDE resmi dalam membangun aplikasi berbasis Android, proses instalasinya juga sederhana sehingga kita dapat dengan mudah membangun aplikasi menggunakan Android Studio



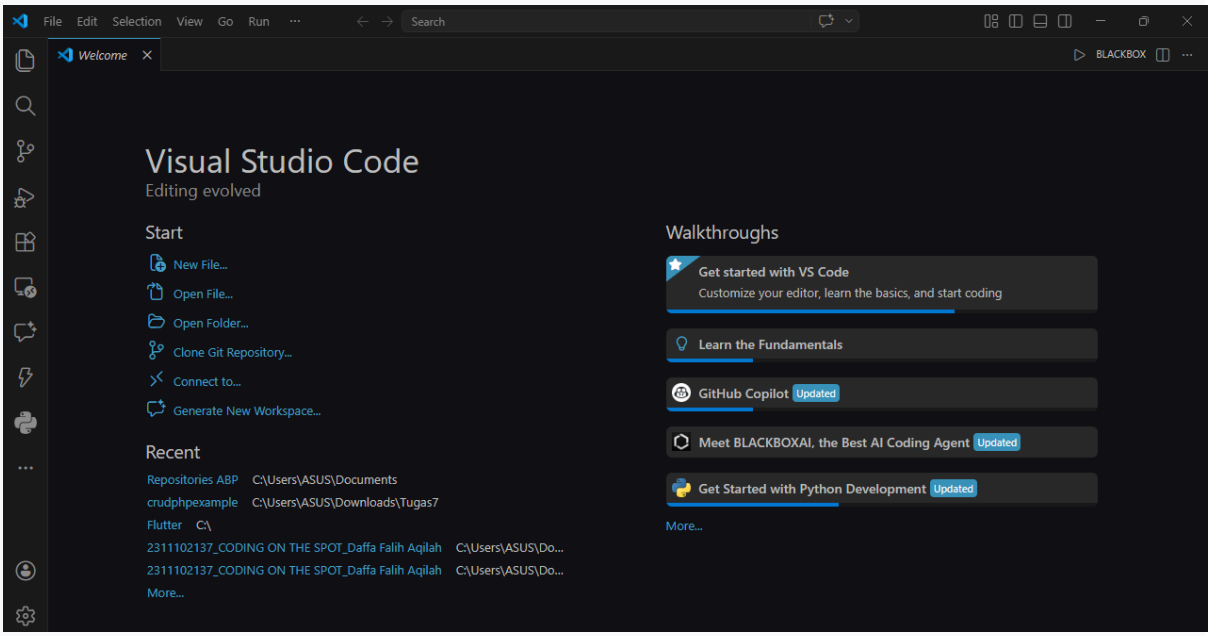
1.5. Instalasi SDK Android

SDK (Standart Development Kit) merupakan kumpulan dari beberapa alat, komponen, juga platform untuk mengembangkan aplikasi berbasis android. SDK wajib ada pada Android Studio.


Settings
Appearance & Behavior
Keymap
Editor
Plugins
Version Control
Build, Execution, Deployment
Languages & Frameworks
C/C++
Android SDK
JVM Logging
Kotlin
Markdown
Schemas and DTDs
Tools
Backup and Sync
Advanced Settings
Espresso Test Recorder
Experimental
Studio Labs
Languages & Frameworks › Android SDK
Manager for the Android SDK and Tools used by the IDE
Android SDK Location: C:\Users\ASUS\AppData\Local\Android\Sdk
Edit
Optimize disk spa
SDK Platforms
SDK Tools
SDK Update Sites
Each Android SDK Platform package includes the Android platform and sources pertaining to an API level by default. Once installed, the IDE will automatically check for updates. Check "show package details" to display individual SDK components.
Name
API Level
Revision
Status
Android CinnamonBun Preview 37.1-beta1 1 Not installed
Android CANARY Preview canary-20260316 11 Not installed
Android 17.0 ("CinnamonBun") 37.0 1 Not installed
Android 16.0 ("Baklava") 36.1 1 Installed
Android Baklava Preview Baklava 1 Not installed
Android 16.0 ("Baklava") 36.0 2 Partially installed
Android 16.0 ("Baklava") 36.0-ext18 1 Not installed
Android 16.0 ("Baklava") 36.0-ext19 1 Not installed
Android 15.0 ("VanillaIceCream") 35 2 Not installed
Android 15.0 ("VanillaIceCream") 35-ext14 1 Not installed
Android 15.0 ("VanillaIceCream") 35-ext15 1 Not installed
Hide Obsolete Packages
Show Package Detai
Project-level settings will be applied to new projects
OK
Cancel
Apply

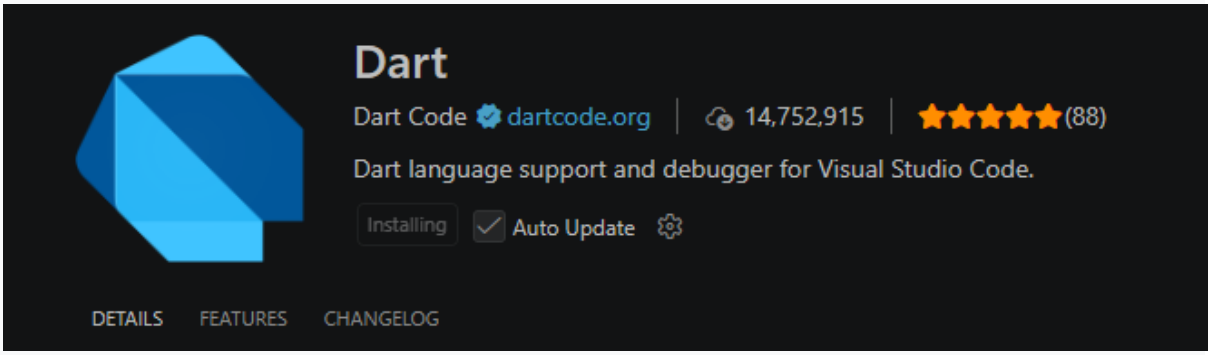
1.6. Instalasi Visual Studio Code

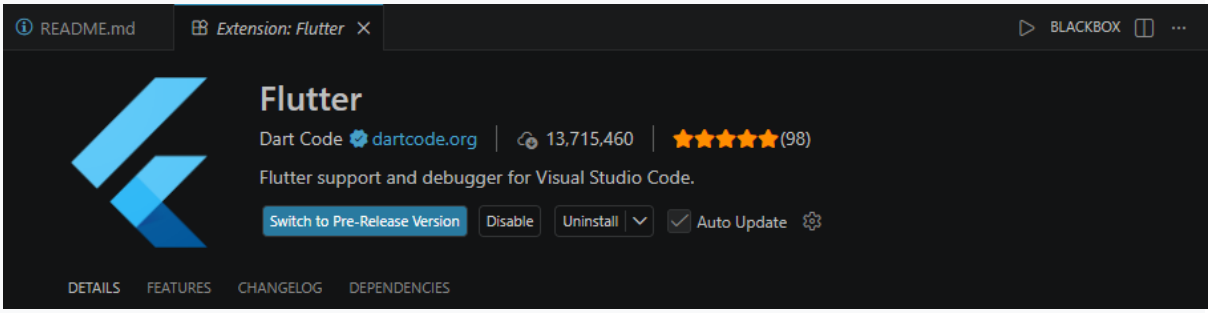
Pada kali ini, kita akan menggunakan visual studio code sebagai editor untuk membuat aplikasi menggunakan Flutter.



1.7. Instalasi Extension Visual Studio Code

Sebelum menggunakan visual studio code, sangat diperlukan untuk menginstall extension yang nantinya sebagai pendukung ketika membuat aplikasi menggunakan Flutter.



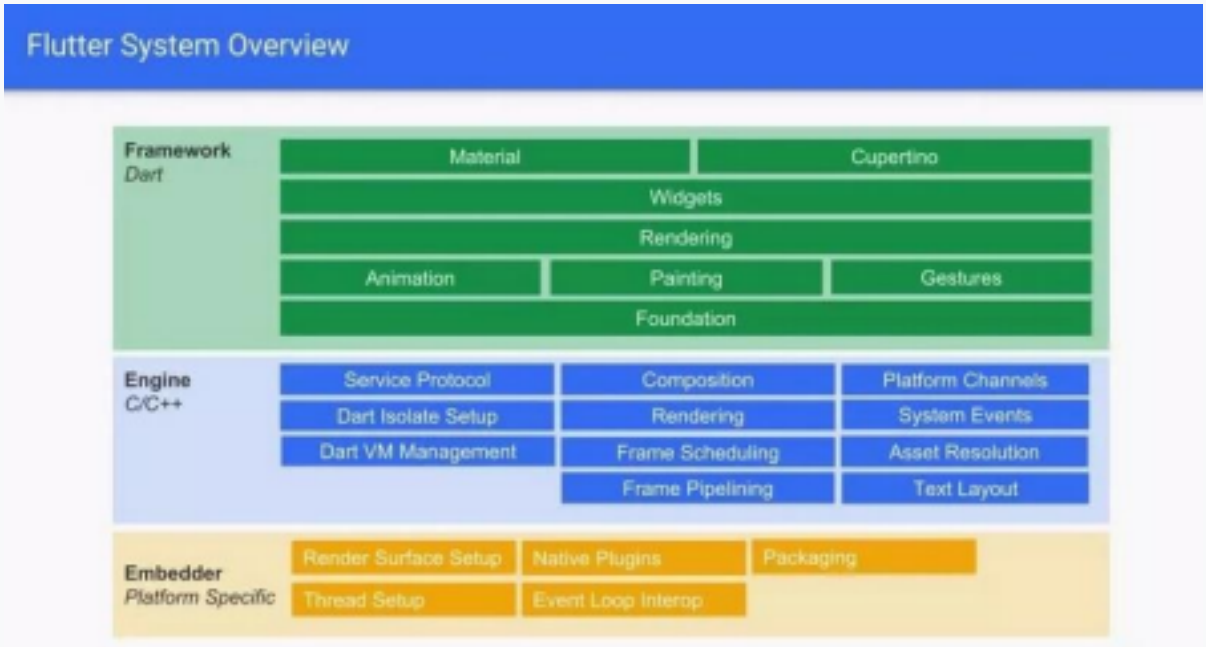


MODUL 2

PENGENALAN FLUTTER

2.1. Apa Itu Flutter

Flutter ditulis menggunakan bahasa C, C++ dan Dart dengan Google's Skia Graphics Engine untuk user interface. Engine yang digunakan untuk produk ini dikenal seperti Google Chrome, Chrome OS, Chromium OS, Mozilla Firefox, Mozilla Thunderbird, Android, Firefox OS dan sekarang Flutter. Flutter berjalan menggunakan Dart Virtual Machine (VM) di sistem operasi Windows, Linux, dan macOS. Dart VM menggunakan kompilasi kode just-in-time (JIT) yang menyediakan fitur hot-reload untuk menghemat waktu pengembangan



Flutter API menggabungkan beberapa widget sesuai dengan kebutuhan pada aplikasi. Konsep widget tree pada dasarnya adalah implementasi widget yang di dalamnya terdapat widget lain yang mewakili komponen antarmuka. Widget bisa stateless atau stateful dan perbedaan dari keduanya sesuai dengan status widget itu sendiri. Hal tersebut berguna untuk membantu mengelola status aplikasi.

2.2. Arsitektur Flutter

Selain dari konsep inti yang dibahas pada sub bab sebelumnya, Flutter mempunyai arsitektur dasar yang nantinya dapat di terapkan pada aplikasi dan mengelola statusnya dengan mudah. Arsitektur yang digunakan pada Flutter disebut Business Logic Component (BLOC). Pada dasarnya, hall ini melalui pendekatan ketika terjadinya suatu event dan menghandle perubahan state pada aplikasi. BLOC merupakan pendekatan yang baik untuk memisahkan logika bisnis dari antarmuka. Ide intinya yang digunakan pada arsitektur BLOC adalah

simplicity, scalability, dan testability, dan semua tujuan ini pasti dicapai dalam arsitektur BLOC.

2.3. Hello World Pada Flutter

Pada pengenalan Flutter kali ini, kita akan membuat Hello World sebagai permulaan ketika menggunakan Flutter.

```
// ignore_for_file: prefer_const_constructors

import 'package:flutter/material.dart';


void main() {

  runApp(const MyApp());

}


class MyApp extends StatelessWidget {

  const MyApp({Key? key}) : super(key: key);


  // This widget is the root of your application.

  @override

  Widget build(BuildContext context) {

    return MaterialApp(

      title: "Hello World",

      home: const MyHomePage(

        title: "Flutter Hello World Page",

      ),

    );

  }

}
```

```
class MyHomePage extends StatefulWidget {
  const MyHomePage({
    Key? key,
    required this.title,
  }) : super(key: key);

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
      body: Center(
        child: Text(
          'Hello World',
        ),
      ),
```

```
    );

  }

}
```

MODUL 3

PENGENALAN DART

3.1. Pengenalan Dart

Untuk belajar flutter, tidak perlu terlalu fasih untuk mempelajari bahasa dart. Terdapat fundamental yang perlu dipelajari seperti variable, statement control, looping, array, fungsi, dsb. Karakteristik bahasa dart mirip dengan bahasa C ataupun Java. Wajib menggunakan titik koma diakhir codingan.

3.1.1. Variable

Untuk penggunaan variable di dart, terdapat beberapa cara, yaitu dengan var, type annotation dan multiple variable.

```
// var

var <variable_name>;

var <name> = <expression>;

// type annotation

<type> <variable_name>;

<type> <name> = <expression>;

// multiple variable

<type> <var1,var2...varN>;
```

Variable primitif yang tersedia di dart :

1. Integer

2. Double

3. String

4. Boolean

3.1.2. Statement Control

Terdapat beberapa cara untuk mendeklarasikan statement control, yaitu if, if else, if else if, switch case.

IF Statement

```
// IF STATEMENT

if (condition) {

  // statements

}
```

IF ELSE Statement

```
// IF ELSE STATEMENT

if (condition) {

  // statements

} else {

  // statements

}
```

IF ELSE IF Statement

```
// IF ELSE IF STATEMENT

if (condition1) {

  // statement(s)

} else if (condition2) {
```

```
  // statement(s)

} else if (conditionN) {

  // statement(s)

} else {

  // statement(s)

}
```

SWITCH CASE Statement

```
// SWITCH CASE

switch (expression) {

  case value1: {

    // statements

  }

  break;




  case value2: {

    // statements

  }

  break;




  default: {

    // statements
```

```
  }

  break;

}
```

3.1.3. Looping

Secara umum, terdapat dua cara untuk melakukan looping di Dart, yaitu menggunakan for loop dan while loop.

For Loops

Gunakan for loop saat kondisinya tahu persis seberapa banyak looping akan dilakukan.

```
for (initial_count_value; termination_condition; step) {

  // statements

}
```

While Loops

Gunakan while loop saat tidak tahu kapan perulangan akan berhenti.

```
while (expression) {

  // statements

}
```

3.1.4. List

Secara umum, kumpulan banyak data dalam satu variabel disebut array, tetapi dalam Dart disebut list. Terdapat dua tipe list:

Fixed Length List

Memiliki panjang tetap dan tidak bisa bertambah.

```
// Mendeklarasikan list

var list_name = new List(initial_size);
```

```
// Menginisialisasikan list

list_name[index] = value;




// Contoh

var newList = new List(3);

newList[0] = 12;

newList[1] = 13;

newList[2] = 11;
```

Growable List

Digunakan jika jumlah data tidak menentu.

```
// Mendeklarasikan list

var list_name = new List(initial_size);




// Menginisialisasikan list

list_name[index] = value;




// Contoh

var newList = new List(3);

newList[0] = 12;
```

```
newList[1] = 13;

newList[2] = 11;
```

3.1.5. Fungsi

Dalam OOP, fungsi memiliki peranan penting untuk menjaga kualitas kode dengan prinsip seperti SOLID, KISS, YAGNI, dan separation of concern.

```
Mendefinisikan Fungsi

void function_name() {

  // statements

}




Memanggil Fungsi

void main() {

  print(factorial(6));

}




Mengembalikan Nilai

factorial(number) {

  if (number <= 0) {

    // termination case

    return 1;

  } else {

    return (number * factorial(number - 1));
```

```
  }

}



Menambahkan Parameter

factorial(number) {

  if (number <= 0) {

    // termination case

    return 1;

  } else {

    return (number * factorial(number - 1));

  }

}
```

Pada fungsi di atas, number merupakan parameter, yaitu variabel dari luar yang digunakan di dalam fungsi.